

LabHW2: Custom tool + two-agent workflow (submission)

Student: Matthew Tirelli

Assignment: Custom tool, Agent 1 (tool cohort), Agent 2 (clinical synopsis).

Folder: 07_rag/LabHW2/

What this PDF contains

Section 1 — Local helper module (*functions.py*): *agent*, *agent_run*, *df_as_text*, Ollama HTTP client, stack-walk tool dispatch, and recovery when models emit JSON-shaped tool text in *message.content* instead of native *tool_calls*.

Section 2 — Main workflow (*labhw2_two_agent_patients.py*): SQLite tool *list_phq9_elevated_with_safety_concerns*, tool metadata, Agent 1 + Agent 2 prompts, and writes to the two markdown deliverables below.

Section 3 — Agent 1 output (*agent1_cohort_findings.md*): Run metadata, cohort counts, and the full markdown table of qualifying visits (PHQ-9 > 15 and safety_concerns = Y).

Section 4 — Agent 2 output (*clinical_findings_synopsis.md*): One-page-style clinical synopsis generated by the second agent from the cohort summary.

Section 1 — functions.py (LabHW2 copy: Ollama + tool execution)

Helper module used by the main script. Defines how tools are resolved in the caller script and how assistant text that looks like a JSON tool call is recovered when needed.

File: functions.py

```
# functions.py
# LabHW2 – Function calling helpers (self-contained copy)
# Adapted from 08_function_calling/functions.py for this assignment folder.
# Pairs with course function-calling materials / functions.R
# Tim Fraser

# Provides agent(), agent_run(), and df_as_text() for Ollama chat + tool execution
.
# Tool implementations live in the caller script; this module dispatches by function name.

# 0. SETUP #####

## 0.1 Load Packages #####

import inspect
import json # for working with JSON
import os
import re
from pathlib import Path

import pandas as pd # for data manipulation
import requests # for HTTP requests

# If you haven't already, install these packages...
# pip install requests pandas tabulate # tabulate: optional dependency for to_markdown

## 0.2 Configuration #####

# Default model and Ollama connection
DEFAULT_MODEL = "smollm2:1.7b"
# Override if Ollama runs on another host/port: export OLLAMA_HOST=http://127.0.0.1:11434

_ollama_port = os.environ.get("OLLAMA_PORT", "11434")
OLLAMA_HOST = os.environ.get("OLLAMA_HOST", f"http://127.0.0.1:{_ollama_port}")
CHAT_URL = f"{OLLAMA_HOST}/api/chat"

# Fail fast if Ollama is down; allow long generation for slow models
REQUEST_TIMEOUT = (10, 600)

def _post_chat(body):
    """POST to Ollama /api/chat with timeout; raise RuntimeError if unreachable."""
    try:
        return requests.post(CHAT_URL, json=body, timeout=REQUEST_TIMEOUT)
    except requests.exceptions.ConnectionError as e:
        raise RuntimeError(
            f"Cannot connect to Ollama at {OLLAMA_HOST}. "
            "Start it (e.g. run `ollama serve` in a terminal) and ensure the model "
            "is pulled."
        ) from e
    except requests.exceptions.Timeout as e:
        raise RuntimeError(
            "Ollama request timed out. The model may be loading or overloaded; try "
            "again."
        ) from e

_FUNCTIONS_FILE = Path(__file__).resolve()

def _globals_for_tool_dispatch():
    """
    Tool callables live in the *script that called agent_run*, not in this module.
    inspect.currentframe().f_back points at agent_run() here, so we must walk up
    until we leave functions.py – otherwise every tool lookup misses and Ollama
    """
```

```

returns empty content.
"""
frame = inspect.currentframe().f_back
while frame is not None:
    gpath = frame.f_globals.get("__file__")
    if gpath:
        try:
            if Path(gpath).resolve() != _FUNCTIONS_FILE:
                return frame.f_globals
        except OSError:
            pass
        frame = frame.f_back
    return globals()

def _parse_tool_arguments(raw_args):
    if raw_args is None:
        return {}
    if isinstance(raw_args, dict):
        return raw_args
    if isinstance(raw_args, str):
        s = raw_args.strip() or "{}"
        try:
            return json.loads(s)
        except json.JSONDecodeError:
            return {}
    return {}

def _resolve_tool_function(caller_globals, func_name):
    if not func_name:
        return None
    fn = caller_globals.get(func_name) or globals().get(func_name)
    if fn is not None:

```

```

        return fn
    for key, val in caller_globals.items():
        if callable(val) and key.lower() == func_name.strip().lower():
            return val
    return None

```

```

def _normalize_embedded_tool_params(raw):
    if raw is None:
        return {}
    if isinstance(raw, dict):
        return raw
    if isinstance(raw, str):
        s = raw.strip()
        if not s:
            return {}
        try:
            return json.loads(s)
        except json.JSONDecodeError:
            return {}
    return {}

```

```

def _recover_tool_output_from_text_content(content, tools, caller_globals):
    """
    Some models (e.g. llama3.2) return a JSON-shaped tool description in message.c
    ontent
    instead of populating message.tool_calls. If it matches a registered tool name
    , run it.

    Returns
    -----
    tuple[str, Any] | None
    (tool_name, return_value) or None
    """

```

```

    """
    if not (content and tools):
        return None

    text = content.strip()
    if text.startswith("```"):
        text = re.sub(r"^```(?:json)?\s*", "", text, flags=re.I)
        text = re.sub(r"\s*```\s*$", "", text)

    registered = []
    for spec in tools:
        fn = (spec.get("function") or {}).get("name")

```

```

if fn:
    registered.append(fn)
if not registered:
    return None

target_name = None
params = {}
if text.startswith("{"):
    try:
        parsed = json.loads(text)
    except json.JSONDecodeError:
        parsed = None
    if isinstance(parsed, dict):
        target_name = parsed.get("name") or parsed.get("tool")
        inner = parsed.get("function")
        if isinstance(inner, dict):
            target_name = inner.get("name") or target_name
            params = _normalize_embedded_tool_params(
                parsed.get("parameters") or parsed.get("arguments")
            )

if not target_name or target_name not in registered:

```

```

for fn in registered:
    if re.search(rf"name\s*:\s*"{re.escape(fn)}", content):
        target_name = fn
        params = {}
        break

if not target_name or target_name not in registered:
    return None

func = _resolve_tool_function(caller_globals, target_name)
if func is None:
    return None

try:
    out = func(**params)
except TypeError:
    try:
        out = func()
    except TypeError:
        return None

return (target_name, out)

def _raise_ollama_ok(response):
    """Turn HTTP errors into actionable messages (404 usually = wrong service on p
ort)."""
    if response.ok:
        return
    snippet = (response.text or "")[:500]
    if response.status_code == 404:
        raise RuntimeError(
            f"HTTP 404 from {response.url}\n"
            "That almost always means nothing at this address is serving Ollama's

```

```

API "
"(another app may be using the port, or OLLAMA_HOST is wrong).\n"
f"Response body (truncated): {snippet!r}\n\n"
"Checks:\n"
f" curl {OLLAMA_HOST}/api/tags\n"
" (should return JSON listing models when Ollama is running)\n"
" lsof -i :11434 # see which process owns the port (macOS/Linux)\n\
n"
"If Ollama uses a different URL, set e.g.:\n"
" export OLLAMA_HOST=http://127.0.0.1:11434"
)
response.raise_for_status()

# 1. AGENT FUNCTION #####

def agent(
    messages,
    model=DEFAULT_MODEL,
    output="text",
    tools=None,
    all=False,
    tool_choice="required",
):

```

```

"""
Agent wrapper function that runs a single agent, with or without tools.

Parameters:
-----
messages : list
List of message dictionaries with 'role' and 'content' keys.
Must follow format: [{"role": "system", "content": "..."}, ...]
model : str
The model to be used for the agent (default: "smollm2:1.7b")
output : str

```

```

The output format (default: "text")
tools : list, optional
List of tool metadata dictionaries for function calling
all : bool
If True, return all responses. If False, return only the last response.
tool_choice : str, dict, or None
When using tools: passed to Ollama as tool_choice (e.g. "required" or a
function dict). None omits the field (older Ollama). Default "required".

Returns:
-----
str or list
The agent's response(s)
"""

```

```

# If the agent has NO tools, perform a standard chat
if tools is None:
    body = {
        "model": model,
        "messages": messages,
        "stream": False,
        "options": {"num_predict": 2048},
    }

    response = _post_chat(body)
    _raise_ollama_ok(response)
    result = response.json()

    return result["message"]["content"]

# If the agent has tools, perform a tool call
body = {
    "model": model,
    "messages": messages,

```

```

    "tools": tools,
    "stream": False,
}
if tool_choice is not None:
    body["tool_choice"] = tool_choice

response = _post_chat(body)
_raise_ollama_ok(response)
result = response.json()

msg = result.get("message") or {}
tool_calls = msg.get("tool_calls") or []

if tool_calls:
    caller_globals = _globals_for_tool_dispatch()
    for tool_call in tool_calls:
        fn_block = tool_call.get("function") or {}
        func_name = fn_block.get("name") or tool_call.get("name")
        raw_args = fn_block.get("arguments", {})
        func_args = _parse_tool_arguments(raw_args)

        func = _resolve_tool_function(caller_globals, func_name)
        if func is None:
            raise RuntimeError(
                f"Model requested unknown tool {func_name!r}. "
                "Define a same-named function in the script that calls agent_r"
            )
        tool_result = func(**func_args)
        tool_call["output"] = tool_result

if all:
    return result

```

```

if tool_calls:
    if output == "tools":
        return tool_calls
    last_out = tool_calls[-1].get("output")
    if last_out is not None:
        return last_out
    return msg.get("content") or ""

# Model put a fake JSON tool call in .content instead of native tool_calls (common w/ llama3.2)
if tools:
    caller_globals = _globals_for_tool_dispatch()
    recovered = _recover_tool_output_from_text_content(msg.get("content") or "", tools, caller_globals)
    if recovered is not None:
        rname, rval = recovered
        if output == "tools":
            return [
                {
                    "function": {"name": rname, "arguments": "{}"},
                    "output": rval,
                }
            ]
        return rval

return msg.get("content") or ""

def agent_run(
    role,
    task,
    tools=None,
    output="text",
    model=DEFAULT_MODEL,

```

```

    tool_choice="required",
):
    """
    Run an agent with a specific role and task.

    Parameters:
    -----
    role : str
        The system prompt defining the agent's role
    task : str
        The user message/task for the agent
    tools : list, optional
        List of tool metadata for function calling
    output : str
        Output format (default: "text")
    model : str
        Model to use (default: DEFAULT_MODEL)
    tool_choice : str, dict, or None
        Ollama tool_choice when tools is set (default "required"). None to omit.

    Returns:
    -----
    str
        The agent's response
    """

    messages = [
        {"role": "system", "content": role},
        {"role": "user", "content": task},
    ]

    if tools is None:
        resp = agent(
            messages=messages,

```

```

            model=model,
            output=output,
            tools=None,
        )
    else:
        resp = agent(
            messages=messages,
            model=model,
            output=output,
            tools=tools,
            tool_choice=tool_choice,
        )

```

```
return resp

# 2. DATA CONVERSION FUNCTION #####

def df_as_text(df):
    """
    Convert a pandas DataFrame to a markdown table string.

    Parameters:
    -----
    df : pandas.DataFrame
    The DataFrame to convert to text

    Returns:
    -----
    str
    A markdown-formatted table string
    """

    tab = df.to_markdown(index=False)
    return tab
```

Section 2 — labhw2_two_agent_patients.py (main submission script)

Custom SQLite tool, tool JSON schema, two-agent chain via `agent_run()`, and paths to `patients.db` and output markdown files.

File: labhw2_two_agent_patients.py

```
# labhw2_two_agent_patients.py
# LabHW2: Custom tool + two-agent workflow (PHQ-9 and safety flags)
# Uses local functions.py in this folder (no imports from 08_function_calling).

# Task 1: Custom SQLite tool + tool metadata for cohort query.
# Task 2: Agent 1 calls the tool; Agent 2 writes a one-page clinical synopsis from
the cohort.
# Task 3: Run, then refine tool descriptions or prompts as needed.

# 0. SETUP #####

## 0.1 Load Packages #####

import sqlite3 # for querying patients.db
import sys
from pathlib import Path

import pandas as pd # for data manipulation

# If you haven't already, install these packages...
# pip install pandas requests tabulate

## 0.2 Import helpers from this folder #####

# Allow `python path/to/labhw2_two_agent_patients.py` from any working directory
LAB_DIR = Path(__file__).resolve().parent
if str(LAB_DIR) not in sys.path:
    sys.path.insert(0, str(LAB_DIR))

from functions import OLLAMA_HOST, agent_run, df_as_text # noqa: E402

## 0.3 Configuration #####

# Use a model you have pulled (`ollama list`). Run: ollama pull llama3.2
```

```
MODEL = "llama3.2"
DB_PATH = LAB_DIR / "patients.db"
OUT_SYNOPSIS = LAB_DIR / "clinical_findings_synopsis.md"
OUT_AGENT1 = LAB_DIR / "agent1_cohort_findings.md"

# PHQ-9 strictly above 15 (i.e., scores 16+)
PHQ9_MIN_EXCLUSIVE = 15

# 1. CUSTOM TOOL #####

def list_phq9_elevated_with_safety_concerns(**kwargs):
    """
    Return visits where PHQ-9 is above the threshold and safety_concerns is Y.

    Always reads LabHW2/patients.db next to this script. Extra tool arguments from
    the model are ignored so a bad db_path cannot open an empty SQLite file (which
    would raise "no such table: visits").

    Returns
    -----
    pandas.DataFrame
    One row per qualifying visit with patient demographics and visit fields.
    """
    path = str(DB_PATH.resolve())

    sql = """
    SELECT
    p.id AS patient_id,
    p.name AS patient_name,
    p.date_of_birth,
    v.id AS visit_id,
    v.visit_date,
    v.phq9_score,
    v.safety_concerns,
```



```

v.diagnosis,
v.provider,
v.medications
FROM visits v
INNER JOIN patients p ON p.id = v.patient_id
WHERE v.phq9_score > ?
AND UPPER(TRIM(COALESCE(v.safety_concerns, ''))) = 'Y'
ORDER BY v.visit_date DESC, p.id ASC;
"""

with sqlite3.connect(path) as conn:
df = pd.read_sql_query(sql, conn, params=(PHQ9_MIN_EXCLUSIVE,))

return df

tool_list_phq9_safety = {
"type": "function",
"function": {
"name": "list_phq9_elevated_with_safety_concerns",
"description": (
"Query the Lab patients database and return a table of visits where "
"PHQ-9 score is greater than 15 (16 or higher) and safety_concerns is "
"Y."
"Each row is one visit with patient name, DOB, visit date, scores, "
"diagnosis, provider, and medications. Call with an empty object {}."
),
"parameters": {"type": "object", "properties": {}},
},
}

AGENT1_TOOL_NAME = tool_list_phq9_safety["function"]["name"]
# Ollama: force this exact tool (needs a tool-capable model + recent Ollama).

```

```

AGENT1_TOOL_CHOICE = {"type": "function", "function": {"name": AGENT1_TOOL_NAME}}

def _coerce_tool_result_to_dataframe(result):
"""
With output='text', agent_run returns the last tool's return value (a DataFrame).
With output='tools', the return value is a list of tool_calls dicts with 'output'.
"""
if isinstance(result, pd.DataFrame):
return result
if isinstance(result, list) and result:
first = result[0]
if isinstance(first, dict) and isinstance(first.get("output"), pd.DataFrame):
return first["output"]
last = result[-1]
if isinstance(last, dict) and isinstance(last.get("output"), pd.DataFrame):
return last["output"]
return None

# 2. TWO-AGENT WORKFLOW #####

print("Starting LabHW2 workflow...", flush=True)
print(f'Ollama base URL: {OLLAMA_HOST} (override with env OLLAMA_HOST if needed)',
flush=True)
print(f'Ollama model: {MODEL} (ensure `ollama serve` is running and the model is pulled)', flush=True)
print("Calling Agent 1 (tool cohort) – this can take a while on first load...", flush=True)

```

```

role1 = (
"You are a clinical data assistant. You may ONLY satisfy requests by calling the "
f"provided tool `{AGENT1_TOOL_NAME}`. Call it exactly once with arguments: {{{}}}"
)
"(empty JSON object). Do not reply with prose, lists, or invented patients."
)
task1 = (
"Pull every visit where PHQ-9 is above 15 and safety_concerns is Y. "
f"Call `{AGENT1_TOOL_NAME}` with {{{}}}."
)

raw1 = agent_run(

```

```

role=role1,
task=task1,
model=MODEL,
tools=[tool_list_phq9_safety],
output="text",
tool_choice=AGENT1_TOOL_CHOICE,
)

coerced_df = _coerce_tool_result_to_dataframe(raw1)
if coerced_df is None:
    preview = ""
    if isinstance(raw1, str) and raw1.strip():
        preview = raw1.strip()[:400].replace("\n", " ")
        raise SystemExit(
            "\nAssignment requirement: Agent 1 must return data via the registered tool.\n"
            f"Expected tool: {AGENT1_TOOL_NAME}\n"
            "Ollama did not yield a tool DataFrame (tool_choice should force this tool).\n"
            "Try: `ollama pull llama3.2` (or another tool-strong tag), set MODEL, "
            "upgrade Ollama, and re-run.\n"

```

```

f"(Raw response type: {type(raw1).__name__}. Preview: {preview!r})\n"
)

cohort_df = coerced_df

# Status: cohort from native tool_calls and/or JSON-shaped tool text in message.content (see LabHW2 functions.py)
print("--- Agent 1 status ---", flush=True)
print(f" Tool registered for Agent 1: {AGENT1_TOOL_NAME}", flush=True)
print(" Ollama tool_choice: forced; Python runs the tool (native API or text-JSON recovery)", flush=True)
print(" Cohort DataFrame produced: yes", flush=True)
print(f" Cohort rows: {len(cohort_df)}", flush=True)
print("-----", flush=True)

n_patients = (
    int(cohort_df["patient_id"].nunique()) if "patient_id" in cohort_df.columns else len(cohort_df)
)
agent1_md = "\n".join(
    [
        "# LabHW2 – Agent 1 cohort findings",
        "",
        "## Run metadata",
        "",
        f"- **Ollama model:** `{MODEL}`",
        f"- **Tool offered to the model:** `{AGENT1_TOOL_NAME}`",
        "- **Tool execution:** Yes (Ollama `tool_calls` and/or recovery from JSON printed in assistant text)",
        f"- **Database:** `{DB_PATH}`",
        "- **Cohort rule:** PHQ-9 score > 15 and `safety_concerns` = Y",
        "",
        "## Summary",
        "",

```

```

f"- Qualifying visits: **{len(cohort_df)}**",
f"- Unique patients (by `patient_id`): **{n_patients}**",
    ],
    "## Full cohort (all columns)",
    "",
    df_as_text(cohort_df),
    "",
]
)
OUT_AGENT1.write_text(agent1_md, encoding="utf-8")
print(f"Wrote Agent 1 hand-in file: {OUT_AGENT1}", flush=True)

print(
    "Calling Agent 2 (clinical synopsis) – with smollm2 this can take a few minutes...",
    flush=True,
)

summary_cols = [
    "patient_id",
    "patient_name",
    "visit_date",
    "phq9_score",
    "safety_concerns",

```

```
"diagnosis",
"provider",
]
cols_ok = [c for c in summary_cols if c in cohort_df.columns]
cohort_sl = cohort_df[cols_ok] if cols_ok else cohort_df
n_unique = cohort_df["patient_id"].nunique() if "patient_id" in cohort_df.columns
else len(cohort_df)
cohort_table = df_as_text(cohort_sl)

role2 = (
```

```
"You are a clinical chart reviewer writing for a quality committee. "
"Use ONLY the data in the user message. If the table is empty, say so clearly.
"
"Do not fabricate patient identifiers or scores. Write in professional clinical
prose."
)
task2 = (
"Cohort: visits with PHQ-9 > 15 and safety_concerns = Y.\n"
f"- Total qualifying visits: {len(cohort_df)}\n"
f"- Unique patients (by patient_id): {n_unique}\n\n"
"Markdown table (key columns only; medications omitted to save context):\n"
f"{cohort_table}\n\n"
"Produce a one-page synopsis in Markdown with these sections:\n"
"## Cohort overview\n"
"## Safety and symptom severity themes\n"
"## Suggested follow-up (operational, non-prescriptive)\n"
"Keep it concise (about one printed page). End with a short bullet list of "
"data limitations (synthetic data, single-site context, etc.)."
)

synopsis_md = agent_run(role=role2, task=task2, model=MODEL, tools=None, output="text")

# 3. WRITE SYNOPSIS AND SHOW RESULTS #####

OUT_SYNOPSIS.write_text(synopsis_md, encoding="utf-8")

print("Agent 2 done.", flush=True)
print(cohort_df.head())
print()
print(f"Wrote Agent 2 hand-in file: {OUT_SYNOPSIS}")
print()
print("Agent 2 synopsis preview (first 1200 chars):")
print(synopsis_md[:1200])
```

```
if len(synopsis_md) > 1200:
print("...")
```

Section 3 — agent1_cohort_findings.md (Agent 1 deliverable)

Written by the workflow after Agent 1: metadata, cohort rule, visit/patient counts, and the full cohort table (all columns).
Rendered below as wrapped monospace to preserve table structure.

File: agent1_cohort_findings.md

```
# LabHW2 – Agent 1 cohort findings

## Run metadata

- **Ollama model:** `llama3.2`
- **Tool offered to the model:** `list_phq9_elevated_with_safety_concerns`
- **Tool execution:** Yes (Ollama `tool_calls` and/or recovery from JSON printed in assistant text)
- **Database:** `/Users/mjt/Desktop/SYSEN/AIDATA/dsai/07_rag/LabHW2/patients.db`
- **Cohort rule:** PHQ-9 score > 15 and `safety_concerns` = Y

## Summary

- Qualifying visits: **40**
- Unique patients (by `patient_id`): **26**

## Full cohort (all columns)

| patient_id | patient_name | date_of_birth | visit_id | visit_date |
| phq9_score | safety_concerns | diagnosis | provider |
| medications |
|-----|-----|-----|-----|-----|
|-----|-----|-----|-----|-----|
|-----|-----|-----|-----|-----|
| 66 | Tim Keller | 1951-12-22 | 201 | 2026-03-07
| 22 | Y | Generalized Anxiety Disorder | Dr. Bowers
| Aripiprazole |
| 75 | Daniel Smith | 1982-05-29 | 234 | 2026-02-15
| 21 | Y | Generalized Anxiety Disorder | Dr. Gordon
| Lithium |
| 61 | Marcus Perez | 1960-08-16 | 184 | 2026-02-02
| 23 | Y | Panic Disorder | Dr. Murphy
| Mirtazapine, Aripiprazole |
| 59 | Deborah Griffin | 1970-05-21 | 176 | 2026-01-09
```

```
| 23 | Y | Panic Disorder | Dr. Olsen
| None |
| 46 | Sharon Ramsey | 1955-03-19 | 136 | 2025-12-20
| 22 | Y | Social Anxiety Disorder | Dr. Davis
| None |
| 47 | Amber Ferguson | 1987-03-19 | 141 | 2025-12-16
| 22 | Y | Bipolar II Disorder | Dr. Caldwell
| Vyvanse, Venlafaxine |
| 3 | Richard McKnight | 2005-06-15 | 12 | 2025-11-22
| 23 | Y | Major Depressive Disorder | Dr. Gomez
| Sertraline, Adderall |
| 3 | Richard McKnight | 2005-06-15 | 8 | 2025-11-16
| 21 | Y | Social Anxiety Disorder | Dr. Watson
| Lamotrigine, Vyvanse |
| 37 | Robert Hutchinson | 1950-03-24 | 107 | 2025-11-01
| 20 | Y | Social Anxiety Disorder | Dr. Reyes
| Lithium, Vyvanse, Fluoxetine |
| 30 | Jeffrey Williams | 1988-05-31 | 92 | 2025-10-06
| 25 | Y | Obsessive-Compulsive Disorder | Dr. McGrath
| Lamotrigine, Aripiprazole |
| 11 | Nicholas Kim | 1954-04-18 | 33 | 2025-10-04
| 20 | Y | Obsessive-Compulsive Disorder | Dr. Le
| Sertraline, Lamotrigine |
| 59 | Deborah Griffin | 1970-05-21 | 178 | 2025-10-01
| 22 | Y | Persistent Depressive Disorder | Dr. Green
| Bupropion, Adderall |
| 73 | Angela Smith | 2006-07-03 | 224 | 2025-09-07
| 22 | Y | Persistent Depressive Disorder | Dr. Barton
| None |
| 54 | Tammy Williamson | 1982-08-22 | 160 | 2025-08-31
| 25 | Y | Major Depressive Disorder | Dr. Nguyen
| None |
| 61 | Marcus Perez | 1960-08-16 | 186 | 2025-08-16
| 23 | Y | Major Depressive Disorder | Dr. Stewart
```

| None |
| 2 | Max Wilson | 1992-09-22 | 7 | 2025-07-27
| 27 | Y | Generalized Anxiety Disorder | Dr. Brooks
| Lithium |
| 17 | Scott Meyer | 1957-10-17 | 52 | 2025-07-01
| 24 | Y | Bipolar II Disorder | Dr. Yu
| None |
| 94 | Stephanie Dominguez | 1967-03-03 | 290 | 2025-06-27
| 23 | Y | Persistent Depressive Disorder | Dr. Mccarth
y | Lamotrigine, Venlafaxine, Adderall |
| 64 | Katherine Edwards | 1974-10-12 | 194 | 2025-06-23
| 22 | Y | Bipolar II Disorder | Dr. Murillo
| Lamotrigine, Escitalopram |
| 37 | Robert Hutchinson | 1950-03-24 | 109 | 2025-06-14
| 26 | Y | Bipolar II Disorder | Dr. Aguilar
| Lithium |
| 29 | Joshua Brown | 2003-07-06 | 91 | 2025-06-13
| 20 | Y | ADHD | Dr. Rodrigu
ez | Aripiprazole, Adderall, Lamotrigine |
| 16 | Gregory Robertson | 1955-02-09 | 48 | 2025-05-22
| 20 | Y | Panic Disorder | Dr. Kelly
| Fluoxetine, Sertraline, Escitalopram |
| 75 | Daniel Smith | 1982-05-29 | 236 | 2025-05-03
| 21 | Y | Obsessive-Compulsive Disorder | Dr. William
s | Lamotrigine, Fluoxetine |
| 32 | Christian Cole | 1985-06-25 | 95 | 2025-04-19
| 20 | Y | Social Anxiety Disorder | Dr. Cortez
| Fluoxetine, Sertraline, Lithium |
| 51 | Kevin Holmes | 1969-05-01 | 152 | 2025-04-15
| 27 | Y | Social Anxiety Disorder | Dr. Mitchel
l | None |
| 47 | Amber Ferguson | 1987-03-19 | 138 | 2025-03-14
| 20 | Y | Obsessive-Compulsive Disorder | Dr. Lee
| None |

| 24 | Danielle Ward | 1979-09-05 | 74 | 2025-02-23
| 26 | Y | Persistent Depressive Disorder | Dr. Little
| Aripiprazole, Lamotrigine, Escitalopram |
| 16 | Gregory Robertson | 1955-02-09 | 47 | 2024-12-20
| 20 | Y | Obsessive-Compulsive Disorder | Dr. Ramirez
| None |
| 73 | Angela Smith | 2006-07-03 | 226 | 2024-12-15
| 20 | Y | Generalized Anxiety Disorder | Dr. Cooley
| Bupropion, Fluoxetine, Mirtazapine |
| 24 | Danielle Ward | 1979-09-05 | 73 | 2024-12-04
| 22 | Y | Social Anxiety Disorder | Dr. Rodrigu
ez | Lamotrigine, Sertraline, Fluoxetine |
| 11 | Nicholas Kim | 1954-04-18 | 35 | 2024-11-12
| 25 | Y | Bipolar II Disorder | Dr. Black
| Venlafaxine, Vyvanse |
| 98 | Hector Mccullough | 1979-09-04 | 307 | 2024-08-30
| 21 | Y | Major Depressive Disorder | Dr. Fox
| Venlafaxine, Bupropion, Fluoxetine |
| 71 | John Molina | 1976-11-19 | 219 | 2024-08-17
| 21 | Y | Major Depressive Disorder | Dr. Hudson
| None |
| 54 | Tammy Williamson | 1982-08-22 | 158 | 2024-08-12
| 20 | Y | Bipolar II Disorder | Dr. Taylor
| Fluoxetine, Sertraline, Mirtazapine |
| 46 | Sharon Ramsey | 1955-03-19 | 135 | 2024-07-17
| 21 | Y | Panic Disorder | Dr. Moreno
| Bupropion |
| 4 | Kevin Diaz | 1998-06-16 | 14 | 2024-06-11
| 24 | Y | Generalized Anxiety Disorder | Dr. Garcia
| Quetiapine |
| 61 | Marcus Perez | 1960-08-16 | 187 | 2024-06-10
| 20 | Y | Generalized Anxiety Disorder | Dr. Herman
| Vyvanse |
| 11 | Nicholas Kim | 1954-04-18 | 36 | 2024-06-09

| 26 | Y | ADHD | Dr. Rivas
| Venlafaxine |
| 60 | Richard Obrien | 1978-05-05 | 181 | 2024-05-01
| 20 | Y | Panic Disorder | Dr. Hall
| Quetiapine, Aripiprazole |
| 14 | Morgan Livingston | 1990-03-19 | 42 | 2024-04-16
| 22 | Y | Major Depressive Disorder | Dr. Wiley
| Aripiprazole, Escitalopram, Bupropion |

Section 4 — clinical_findings_synopsis.md (Agent 2 deliverable)

Markdown synopsis from Agent 2 (clinical chart review style) based on the cohort passed in the prompt.

File: clinical_findings_synopsis.md

```
# Clinical Chart Review Synopsis
## Cohort Overview
The cohort consists of 40 unique patients with PHQ-9 scores > 15 and safety concerns. The patients have been diagnosed with various mental health conditions, including Generalized Anxiety Disorder, Panic Disorder, Bipolar II Disorder, Major Depressive Disorder, Social Anxiety Disorder, Obsessive-Compulsive Disorder, ADHD, and Persistent Depressive Disorder.

## Safety and Symptom Severity Themes
The data highlights several safety concerns and symptom severity themes:
* **Generalized Anxiety Disorder**: Patients with Generalized Anxiety Disorder have high PHQ-9 scores (> 20) and a higher rate of safety concerns (Y).
* **Panic Disorder**: Patients with Panic Disorder have moderate to high PHQ-9 scores (> 20) and a higher rate of safety concerns (Y).
* **Bipolar II Disorder**: Patients with Bipolar II Disorder have moderate to high PHQ-9 scores (> 20) and a higher rate of safety concerns (Y).
* **Major Depressive Disorder**: Patients with Major Depressive Disorder have moderate to high PHQ-9 scores (> 20) and a higher rate of safety concerns (Y).
* **Social Anxiety Disorder**: Patients with Social Anxiety Disorder have moderate PHQ-9 scores (> 20) and a higher rate of safety concerns (Y).

## Suggested Follow-up
Based on the data, it is recommended to:
* Conduct thorough mental health assessments for patients with Generalized Anxiety Disorder, Panic Disorder, Bipolar II Disorder, Major Depressive Disorder, and Social Anxiety Disorder.
* Implement safety protocols for patients with high PHQ-9 scores (> 25) and a history of safety concerns (Y).
* Monitor patients closely for symptom severity changes and adjust treatment plans as needed.

## Data Limitations
The data has several limitations:
* **Synthetic Data**: The data may not accurately represent real-world patient populations due to its synthetic nature.
* **Single-Site Context**: The data is limited to a single site, which may not provide a representative sample of mental health patients across different healthcare settings.
* **Limited Demographic Information**: The data lacks detailed demographic information, such as age, sex, and socioeconomic status.
```